NAME: K.JANANI

REGISTER NO: 192511243

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

ASSESSMENT TOOL 1

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Code of Conduct:

I K JANANI (192511243) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

1.Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.

Relational Algebra Queries:

Assume:

- Employee(EmpID, EmpName, DeptID, Salary)
- Department(DeptID, DeptName, Location) **1.**

Selection (σ)

Find employees with salary greater than 50,000:

$$\sigma_{Salary>50000}(Employee)$$

2. Projection (π)

List only employee names and salaries:

$$\pi_{EmpName,Salary}(Employee)$$

3. Union (∪)

Get all department IDs appearing in either Employee or Department:

$$\pi_{DeptID}(Employee) \cup \pi_{DeptID}(Department)$$

4. Difference (-)

Find department IDs that exist in Department but have no employees:

$$\pi_{DeptID}(Department) - \pi_{DeptID}(Employee)$$

5. Cartesian Product (×)

Combine every employee with every department (not meaningful alone, but useful before join):

$$Employee \times Department$$

6. Join (⋈)

Get employee names with their department names:

$$Employee \bowtie_{Employee.DeptID=Department.DeptID} Department.$$

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

```
mysql> use a;
Database changed
mysql> create table STUDENT(SID int,Name varchar(30),DOB date,DeptID int);
Query OK, 0 rows affected (0.36 sec)

mysql> create table DEPARTMENT(DeptID int,DeptName varchar(30));
Query OK, 0 rows affected (0.13 sec)

mysql> desc STUDENT;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| SID    | int         | YES  |     | NULL    |       |
| Name   | varchar(30) | YES  |     | NULL    |       |
| DOB    | date        | YES  |     | NULL    |       |
| DeptID | int         | YES  |     | NULL    |       |
+--------+-------------+------+-----+---------+-------+
4 rows in set (0.20 sec)

mysql> desc DEPARTMENT;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptID   | int         | YES  |     | NULL    |       |
| DeptName | varchar(30) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.01 sec)
```

```
mysql> insert into STUDENT values(02,'Akash','2011=04-03',1925),(03,'Bala','2011-04-12',1923),(04,'Banu','2012-05-23'
Query OK, 3 rows affected, 1 warning (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 1

mysql> select * from STUDENT;
+------+-------+------------+--------+
| SID  | Name  | DOB        | DeptID |
+------+-------+------------+--------+
|    1 | Anil  | 2012-09-02 |   1925 |
|    2 | Akash | 2011-04-03 |   1925 |
|    3 | Bala  | 2011-04-12 |   1923 |
|    4 | Banu  | 2012-05-23 |   1921 |
+------+-------+------------+--------+
4 rows in set (0.00 sec)
```

3.Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

CREATE TABLE Sales (SaleID INT PRIMARY KEY,ProductName VARCHAR(50),Category VARCHAR(30),Amount DECIMAL(10,2),Quantity INT);

```
mysql> select * from Sales;
+--------+-------------+-------------+----------+----------+
| SaleID | ProductName | Category    | Amount   | Quantity |
+--------+-------------+-------------+----------+----------+
|    101 | Laptop      | Electronics | 50000.00 |        2 |
|    102 | Mobile      | Electronics | 25000.00 |        5 |
|    103 | Chair       | Furniture   |  8000.00 |        4 |
|    104 | Table       | Furniture   | 12000.00 |        3 |
|    105 | Laptop      | Electronics | 55000.00 |        1 |
|    106 | Chair       | Furniture   |  9000.00 |        2 |
+--------+-------------+-------------+----------+----------+
6 rows in set (0.04 sec)
```

COUNT:

SELECT Category,

COUNT(*) AS Total_Products

FROM Sales

GROUP BY Category;

```
+-------------+----------------+
| Category    | Total_Products |
+-------------+----------------+
| Electronics |              3 |
| Furniture   |              3 |
+-------------+----------------+
2 rows in set (0.01 sec)
```

SUM:

SELECT Category,SUM(Amount) AS Total_Sales FROM Sales GROUP BY Category;

```
mysql> SELECT Category,SUM(Amount) AS Total_Sales FROM Sales GROUP BY Category;
+-------------+-------------+
| Category    | Total_Sales |
+-------------+-------------+
| Electronics |   130000.00 |
| Furniture   |    29000.00 |
+-------------+-------------+
2 rows in set (1.32 sec)
```

AVG:

SELECT Category,AVG(Amount) AS Average_Sales FROM Sales GROUP BY Category;

```
mysql> SELECT Category,AVG(Amount) AS Average_Sales FROM Sales GROUP BY Category;
+-------------+---------------+
| Category    | Average_Sales |
+-------------+---------------+
| Electronics |  43333.333333 |
| Furniture   |   9666.666667 |
+-------------+---------------+
2 rows in set (0.01 sec)
```

MAX:

SELECT Category,MAX(Amount) AS Highest_Sale FROM Sales GROUP BY Category;

```
mysql> SELECT Category,MAX(Amount) AS Highest_Sale FROM Sales GROUP BY Category;
+-------------+--------------+
| Category    | Highest_Sale |
+-------------+--------------+
| Electronics |     55000.00 |
| Furniture   |     12000.00 |
+-------------+--------------+
2 rows in set (0.03 sec)
```

MIN:

SELECT Category,MIN(Amount) AS LOWEST_Sale FROM Sales GROUP BY Category;

```
mysql> SELECT Category,MIN(Amount) AS LOWEST_Sale FROM Sales GROUP BY Category;
+-------------+-------------+
| Category    | LOWEST_Sale |
+-------------+-------------+
| Electronics |    25000.00 |
| Furniture   |     8000.00 |
+-------------+-------------+
2 rows in set (0.01 sec)
```

4.Write a correlated sub-query to find employees earning more than the average salary of their own department.

CREATE TABLE Employee (Emp_ID INT PRIMARY KEY,Emp_Name VARCHAR(50),Salary DECIMAL(10,2),Dept_ID VARCHAR(5));

INSERT INTO Employee VALUES (101,'Ravi',50000,'D1'),(102,'Priya',60000,'D2'), (103,'Karan',45000,'D1'), (104,'Anu',70000,'D3'),(105,'John',55000,'D2'),(106,'Meena',40000,'D3');

```
mysql> select * from Employee;;
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|    101 | Ravi     | 50000.00 | D1      |
|    102 | Priya    | 60000.00 | D2      |
|    103 | Karan    | 45000.00 | D1      |
|    104 | Anu      | 70000.00 | D3      |
|    105 | John     | 55000.00 | D2      |
|    106 | Meena    | 40000.00 | D3      |
+--------+----------+----------+---------+
6 rows in set (0.02 sec)
```

SELECT E.Emp_ID,E.Emp_Name,E.Salary,E.Dept_ID FROM Employee E WHERE E.Salary > (SELECT AVG(Salary) FROM Employee WHERE Dept_ID = E.Dept_ID);

```
mysql> SELECT E.Emp_ID,E.Emp_Name,E.Salary,E.Dept_ID FROM Employee E WHERE E.Salary > (SELECT AVG(Salary) FROM Employee
WHERE Dept_ID = E.Dept_ID);
+--------+----------+----------+---------+
| Emp_ID | Emp_Name | Salary   | Dept_ID |
+--------+----------+----------+---------+
|    101 | Ravi     | 50000.00 | D1      |
|    102 | Priya    | 60000.00 | D2      |
|    104 | Anu      | 70000.00 | D3      |
+--------+----------+----------+---------+
3 rows in set (0.07 sec)
```

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

CREATE TABLE Project (Project_ID VARCHAR(5) VARCHAR(50));

INNER JOIN:

SELECT

E.Emp_ID,

E.Emp_Name,

P.Project_Name

FROM Employee E

INNER JOIN Project P

ONE.Project_ID=P.Project_ID;

```
+--------+----------+--------------------+
| Emp_ID | Emp_Name | Project_Name       |
+--------+----------+--------------------+
|    101 | Ravi     | Inventory System   |
|    102 | Priya    | Hospital Management |
+--------+----------+--------------------+
2 rows in set (0.04 sec)
```

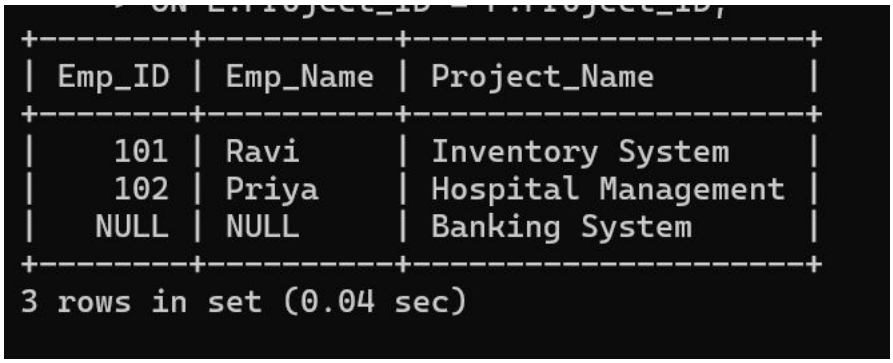
LEFT OUTER JOIN:

```
SELECT
  E.Emp_ID,
  E.Emp_Name,
  P.Project_Name
FROM Employee E
LEFT OUTER JOIN Project P
ON E.Project_ID = P.Project_ID;
```

```
    -> ON E.Project_ID = P.Project_ID;
+--------+----------+--------------------+
| Emp_ID | Emp_Name | Project_Name       |
+--------+----------+--------------------+
|    101 | Ravi     | Inventory System   |
|    102 | Priya    | Hospital Management |
|    103 | Karan    | NULL               |
|    104 | Anu      | NULL               |
|    105 | John     | NULL               |
|    106 | Meena    | NULL               |
+--------+----------+--------------------+
6 rows in set (0.04 sec)
```

RIGHT OUTER JOIN:

```
SELECT
  E.Emp_ID,
  E.Emp_Name,
  P.Project_Name
FROM Employee E
RIGHT OUTER JOIN Project P
ON E.Project_ID = P.Project_ID;
```



```
+--------+----------+--------------------+
| Emp_ID | Emp_Name | Project_Name       |
+--------+----------+--------------------+
|    101 | Ravi     | Inventory System   |
|    102 | Priya    | Hospital Management |
|   NULL | NULL     | Banking System     |
+--------+----------+--------------------+
3 rows in set (0.04 sec)
```

FULL OUTER JOIN:

```
SELECT
  E.Emp_ID,
  E.Emp_Name,
  P.Project_Name
FROM Employee E
LEFT JOIN Project P
ON E.Project_ID = P.Project_ID
UNION
SELECT
  E.Emp_ID,
  E.Emp_Name,
  P.Project_Name
FROM Employee E
RIGHT JOIN Project P
ON E.Project_ID = P.Project_ID;
```

```
+--------+----------+--------------------+
| Emp_ID | Emp_Name | Project_Name       |
+--------+----------+--------------------+
|    101 | Ravi     | Inventory System   |
|    102 | Priya    | Hospital Management |
|    103 | Karan    | NULL               |
|    104 | Anu      | NULL               |
|    105 | John     | NULL               |
|    106 | Meena    | NULL               |
|   NULL | NULL     | Banking System     |
+--------+----------+--------------------+
7 rows in set (0.08 sec)
```

CROSS:

```
SELECT
  E.Emp_Name,
  P.Project_Name
FROM Employee E
CROSS JOIN Project P;
```

```
+----------+--------------------+
| Emp_Name | Project_Name       |
+----------+--------------------+
| Ravi     | Banking System     |
| Ravi     | Hospital Management |
| Ravi     | Inventory System   |
| Priya    | Banking System     |
| Priya    | Hospital Management |
| Priya    | Inventory System   |
| Karan    | Banking System     |
| Karan    | Hospital Management |
| Karan    | Inventory System   |
| Anu      | Banking System     |
| Anu      | Hospital Management |
| Anu      | Inventory System   |
| John     | Banking System     |
| John     | Hospital Management |
| John     | Inventory System   |
| Meena    | Banking System     |
| Meena    | Hospital Management |
| Meena    | Inventory System   |
+----------+--------------------+
18 rows in set (0.00 sec)
```

6. Create a view ‘DeptSalaryView’; that displays department name, total employees, and average salary. Write a query on this view.

```
CREATE TABLE Department(
  DeptID VARCHAR(5) PRIMARY KEY,
  DeptName VARCHAR(50)
);
INSERT INTO Department VALUES ('D1','HR'), ('D2','Finance'), ('D3','IT');
CREATE VIEW DeptSalaryView AS
SELECT
D.DeptName,
COUNT(E.EmpID) AS TotalEmployees,
AVG(E.Salary) AS AverageSalary
FROM Department D
JOIN Employee E
ON D.DeptID=E.DeptID
GROUP BY D.DeptName;
SELECT * FROM DeptSalaryView;
```

```
mysql> SELECT * FROM DeptSalaryView;
+----------+----------------+---------------+
| DeptName | TotalEmployees | AverageSalary |
+----------+----------------+---------------+
| HR       |              2 |  47500.000000 |
| Finance  |              2 |  57500.000000 |
| IT       |              2 |  55000.000000 |
+----------+----------------+---------------+
3 rows in set (0.18 sec)
```

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Relational Algebra

Relations:

STUDENT(SID, Name)

COURSE(CID, CourseName)

ENROLLMENT(SID, CID)

Relational Algebra Expression

πName (STUDENT ⋈ (πSID (ENROLLMENT ÷ πCID(COURSE))))

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

UPDATE:

update Student set Grade='C' where StudentID=1001;

```
mysql> update Student set Grade='C' where StudentID=1001;
Query OK, 1 row affected (0.16 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from Student;
+-----------+-------------+-------+---------+
| StudentID | StudentName | Grade | ClassID |
+-----------+-------------+-------+---------+
|      1001 | Ravi        | C     |     101 |
|      1002 | Priya       | A+    |     102 |
|      1003 | Karan       | B+    |     101 |
|      1004 | Anu         | A     |     103 |
+-----------+-------------+-------+---------+
4 rows in set (0.04 sec)
```

DELETE:

DELETE FROM Student

WHERE StudentID=1003;

```
mysql> select * from Student;
+-----------+-------------+-------+---------+
| StudentID | StudentName | Grade | ClassID |
+-----------+-------------+-------+---------+
|      1001 | Ravi        | C     |     101 |
|      1002 | Priya       | A+    |     102 |
|      1004 | Anu         | A     |     103 |
+-----------+-------------+-------+---------+
3 rows in set (0.01 sec)
```

9.Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Tuple Relational Calculus

Relations:

EMPLOYEE(EmpID, EmpName, Salary, DeptID)

DEPARTMENT(DeptID, DeptName)

Tuple Relational Calculus Expression

{ E.EmpName |

E ∈ EMPLOYEE ∧

∃ D ∈ DEPARTMENT

(D.DeptID = E.DeptID ∧

D.DeptName = 'IT' ∧

E.Salary > 50000)

}

10.Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Create Passenger Table

```
CREATE TABLE Passenger(
PassengerID INT PRIMARY KEY,
PassengerName VARCHAR(50),
Gender VARCHAR(10),
Age INT,
Phone VARCHAR(15)
);
```

Create Flight Table

```
CREATE TABLE Flight(
FlightID VARCHAR(5) PRIMARY
KEY,
FlightName VARCHAR(50),
Source VARCHAR(30),
Destination VARCHAR(30),
DepartureTime TIME
);
```

Create Reservation Table

```
CREATE TABLE Reservation(
```

ReservationID INT PRIMARY KEY,

PassengerID INT,

```
FlightID VARCHAR(5),
JourneyDate DATE,
SeatNo VARCHAR(10),
FOREIGN KEY(PassengerID)
REFERENCES Passenger(PassengerID),
FOREIGN KEY(FlightID)
REFERENCES Flight(FlightID)
);
```

Insert Records

```
INSERT INTO Passenger
VALUES
(101,'Ravi','Male',25,'9876543210'),
(102,'Priya','Female',24,'9876501234'),
(103,'Karan','Male',30,'9123456789');
INSERT INTO Flight
VALUES
('F101','Air India','Chennai','Delhi','10:30:00'),
('F102','IndiGo','Mumbai','Bangalore','14:15:00'),
('F103','SpiceJet','Hyderabad','Kolkata','18:45:00');
INSERT INTO Reservation
VALUES
(1,101,'F101','2026-08-15','A1'),
(2,102,'F102','2026-08-16','B2'),
(3,103,'F101','2026-08-17','A3');
```

Query 1 – INNER JOIN

```
SELECT
P.PassengerName,
F.FlightName,
F.Source,
F.Destination
FROM Passenger P
```

INNER JOIN Reservation R

ON P.PassengerID=R.PassengerID

INNER JOIN Flight F

ON R.FlightID=F.FlightID;

```
+---------------+------------+---------+-------------+
| PassengerName | FlightName | Source  | Destination |
+---------------+------------+---------+-------------+
| Ravi          | Air India  | Chennai | Delhi       |
| Priya         | IndiGo     | Mumbai  | Bangalore   |
| Karan         | Air India  | Chennai | Delhi       |
+---------------+------------+---------+-------------+
3 rows in set (0.03 sec)
```

Query 2 – LEFT JOIN

SELECT

P.PassengerName,

R.ReservationID

FROM Passenger P

LEFT JOIN Reservation R

ON

P.PassengerID=R.PassengerID;

```
+---------------+---------------+
| PassengerName | ReservationID |
+---------------+---------------+
| Ravi          |             1 |
| Priya         |             2 |
| Karan         |             3 |
+---------------+---------------+
3 rows in set (0.01 sec)
```

Query 3 – Subquery

SELECT PassengerName

FROM Passenger

WHERE PassengerID IN

(

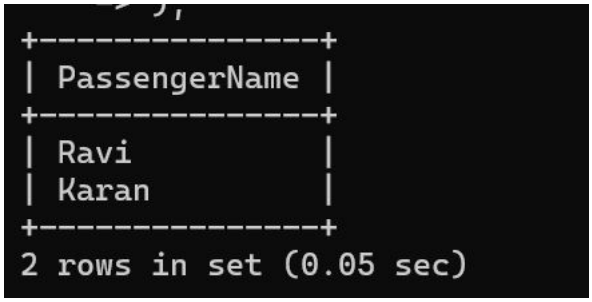
SELECT PassengerID

FROM Reservation

WHERE

FlightID='F101'

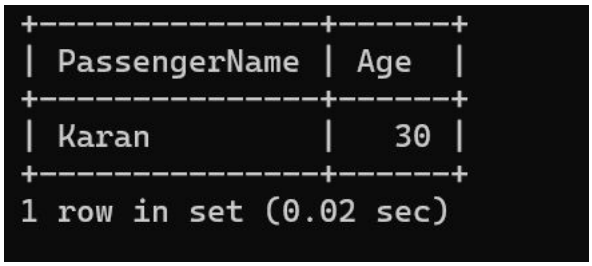
);



```
+---------------+
| PassengerName |
+---------------+
| Ravi          |
| Karan         |
+---------------+
2 rows in set (0.05 sec)
```

Query 4 – Correlated Subquery

```
SELECT PassengerName,Age
FROM Passenger P
WHERE Age >
(
SELECT AVG(Age)
FROM Passenger
);
```



```
+---------------+------+
| PassengerName | Age  |
+---------------+------+
| Karan         |   30 |
+---------------+------+
1 row in set (0.02 sec)
```

Query 5 – View

Create View

```
CREATE VIEW FlightReservationView AS
SELECT
P.PassengerName,
F.FlightName,
F.Source,
F.Destination,
R.SeatNo
FROM Passenger P
JOIN Reservation R
ON P.PassengerID=R.PassengerID
```

JOIN Flight F

ON R.FlightID=F.FlightID;

SELECT * FROM

FlightReservationView;

```
mysql> SELECT * FROM FlightReservationView;
+---------------+------------+---------+-------------+--------+
| PassengerName | FlightName | Source  | Destination | SeatNo |
+---------------+------------+---------+-------------+--------+
| Ravi          | Air India  | Chennai | Delhi       | A1     |
| Priya         | IndiGo     | Mumbai  | Bangalore   | B2     |
| Karan         | Air India  | Chennai | Delhi       | A3     |
+---------------+------------+---------+-------------+--------+
3 rows in set (0.06 sec)
```